Lec-5
04/05/26 7:56 PM

number-System

Data Type and Type-casting

decimal | Binary number System

base-10 → (0 to 9) → 10

2 → 0,1

Octal number 8 ← 0 to 7

46 ✓

2|46 0
2|23 1
2|11 1
2|5 1
2|2 0
2|1 1
0

101110

39 → binary Rep

2|39 1
2|19 1
2|9 1
2|4 0
2|2 0
2|1 1
0

100111

57 0111001

2|57 1
2|28 0
2|14 0
2|7 1
2|3 1
2|1 1
0

1000 1

10000 + 1 = 10001

2|17 1×10^0
2|8 0×10^1
2|4 0×10^2
2|2 0×10^3
2|1 1×10^4
0

10001
= 1×10^4 + 0×10^3 + 0×10^2 + 0×10^1 + 1×10^0

n ✓ int sum = 0, mul = 1

```
while(n>0){
  int rem = n%2
    sum = sum + rem x mul
    mul = mul x10
    n = n/2
}
```

378 = 3×10^2 + 7×10^1 + 8×10^0

int sum = 0, mul = 1 0 5

```
while(n>0){
  int rem = n%2 ✓
    sum = sum + rem x mul
    mul = mul x10 ←
    = n/2 ✓
}
```

2|37 1×10^0
2|18 0×10^1
2|9 1×10^2
2|4 0×10^3
2|2 0×10^4
2|1 1×10^5
0

sum = 1+0+100+0+0+100000
= 100000 + 101
100101

100101

1 0 1 0 1
2^4 2^3 2^2 2^1 2^0

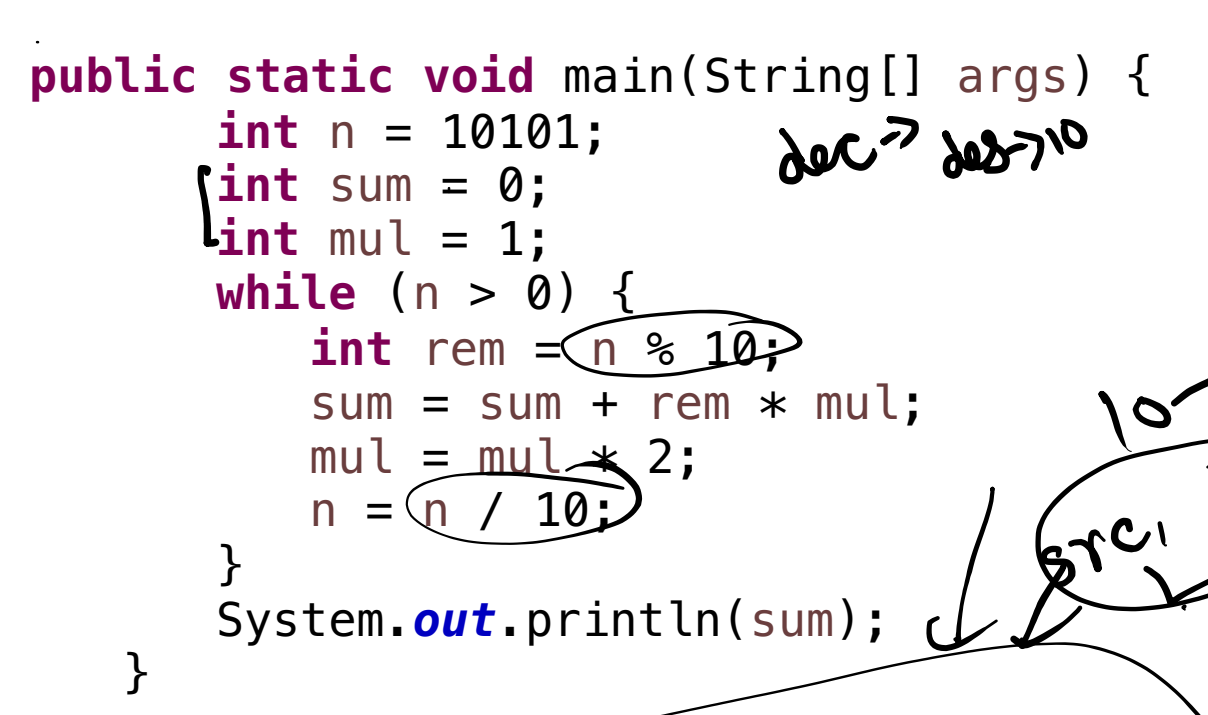
mul = 1

Code

10|1101101
10|110110
10|11011
10|1101
10|110
10|11
10|1
0

1×2^0 = 1
0×2^1 = 0
1×2^1 = 2
1×2^3 = 8
0×2^4 = 0
1×2^5 = 32
1×2^6 = 64
107

```
while(n>0){
  int rem = n%10
    sum = sum+rem mul
    mul = mul*2
    n = n/10
}
```



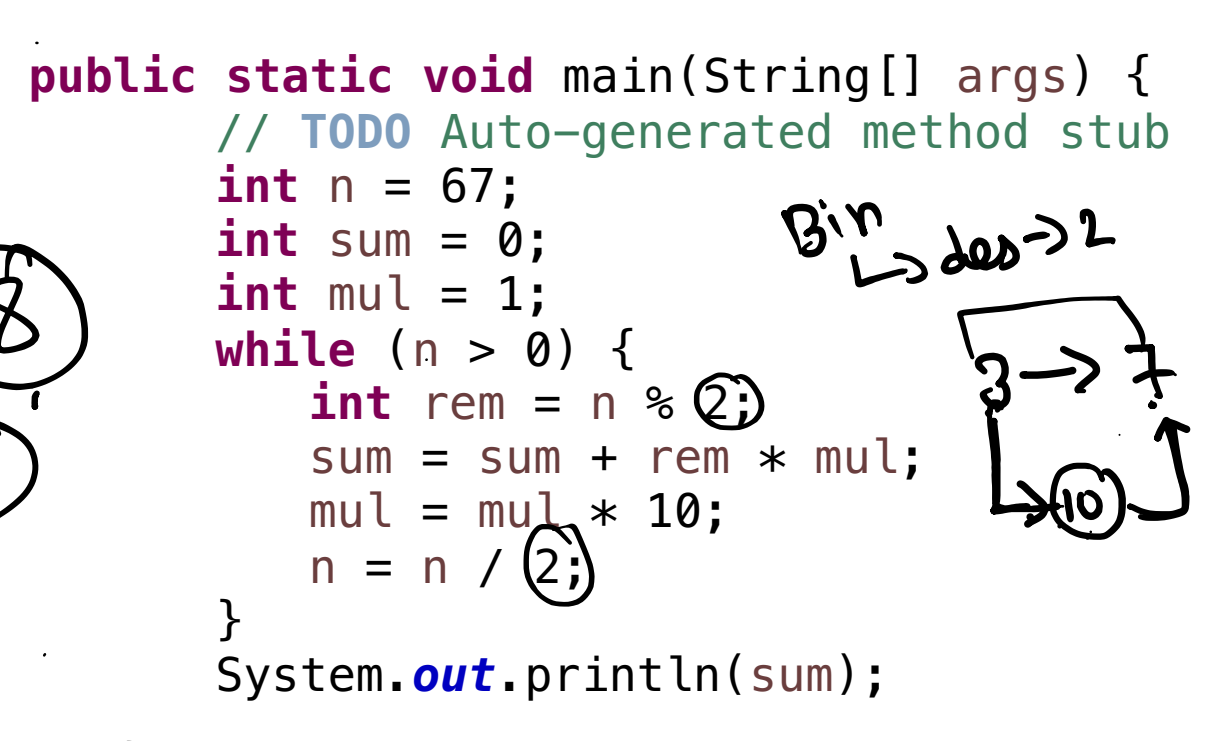
```
public static void main(String[] args) {
		int n = 10101;
		int sum = 0;
		int mul = 1;
		while (n > 0) {
			int rem = n % 10;
			sum = sum + rem * mul;
			mul = mul * 2;
			n = n / 10;
		}
		System.out.println(sum);
	}
```

dec → des→10

10 → 8

src, des 10

```
while(n>0){
  rem = n% des
  sum = sum+rem×mul
  mul = mul * src
  n = n/des
}
```



```
public static void main(String[] args) {
		// TODO Auto-generated method stub
		int n = 67;
		int sum = 0;
		int mul = 1;
		while (n > 0) {
			int rem = n % 2;
			sum = sum + rem * mul;
			mul = mul * 10;
			n = n / 2;
		}
		System.out.println(sum);
	}
```

Bin → des→2

3 → 7 → 10

base 012

10 | 20134 5 10

(0 to 9) % 10 =

12%10
12/10=1
6%10=6
6/10=0
11%10=1
11/10=1
12%10=2
12/10=1

 1 0 1
 2 3 4 5 — 10
+ 9 8 1 7 — 10
1 2 1 6 2

Condition → base → same → sum

%2
/2

1 0 1 1 1
 1 0 1 0 1 1
 1 0 0 1 1 1
1 0 1 0 0 1 0

2%2=0 ✓
2/2=1 ✓
3%2=1
3/2=1
1%2=1
1/2=0

60 20

Data-Type →

primitive — non-primitive

memory

Friday

7 ✓

int x=5

primitive:
- byte
- short
- int
- long

only integer

float, double] floating number

char

boolean

9.7 2.9 6.9 54

101

data Type	Size	Range
byte	8bit → 1byte	
Short	16 bit → 2byte	
int	32bit → 4byte	
Long	64 bit → 8byte	
Float	32 bit → 4byte	
double	64bit → 8byte	
Char	16bit → 2byte	
boolean	1bit 0,1	

bit, byte

1GB → 1024MB → 1024KB → 1mb → Byte → 8 ← Bit

8GB

byte b=5
0000 0101 ✓

int x = 5
00000000…101
2^9 2^0 2^2 2^1 2^0

50, Sel → 60

Range

byte b=

1 1 1 1 1 1 1 1
0 0 0 0 0 0 0 0

count = 256
[0, 255]

+ve ✗

+ve | -ve

0111 1111

127
127 +1
255

[-127, 127]

1000 0000 -0
0000 0000 +0

① sign-bit

byte

✗

-ve ← 1
+ve ← 0

MSB

LSB

256 → 255

1S complement

00000101 → 5
11111010 → -5

01101 011
10010 100

[0000 0000] → 0
1111 1111 → -0 ✗

2S complement

27
byte=27
byte b=-27

00011011 → byte b=27

01101010
10010101
0000 0001
10010110

2S Complm.

111 00100
 1
11100101 → byte b=-27

2S complement

-ve ← 1 000 0000
+ve ← 0 111 1111

MSB

256
[-128, 127]

①000 0000

0 111 1111
0 000 0001
1000 0000 → -128

C8SOT.com

byte → 8bit → -2^{8-1} to $2^{8-1}-1$

byte → 8bit → -2^7 to 2^7-1

Short → 16bit → -2^{15} to $2^{15}-1$

int → 32bit → -2^{31} to $2^{31}-1$

long → 64bit → -2^{63} to $2^{63}-1$

9 digit

18 long

```
yte b = (byte)(428);
```

1
1 1 0 1 0 1 1 0 0

10101100
01010011
 111
01010100 → -84